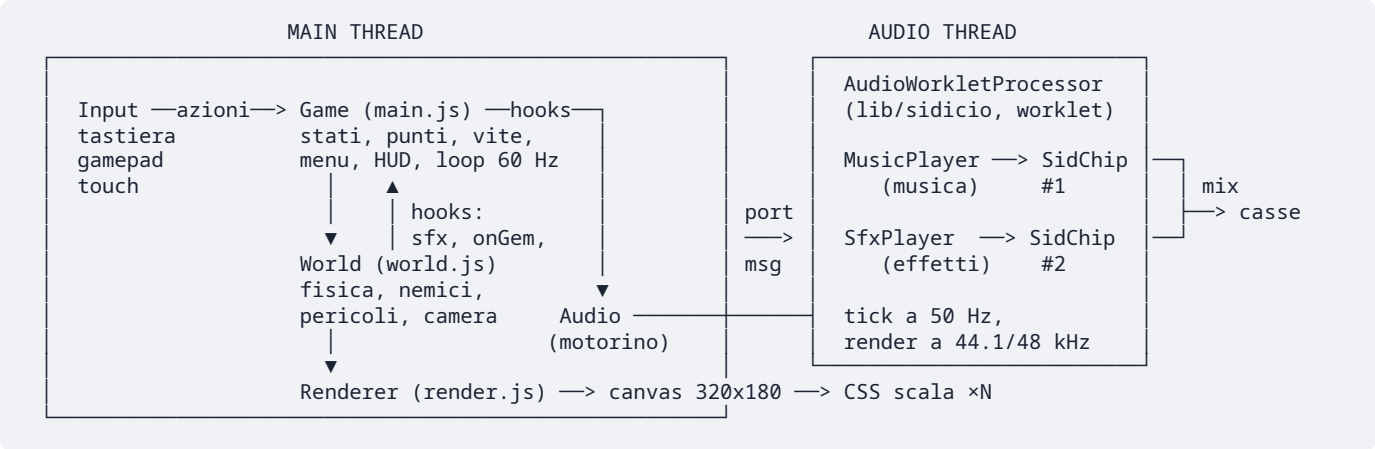
VULCANO

Vulcano — architettura e funzionamento interno

Questo documento spiega come è fatto il gioco dentro: i moduli, come si parlano, e come funziona ciascuno. Per avvio, test e modifica dei contenuti vedi TECNICA.md.

1. Il quadro d'insieme

Il gioco è JavaScript puro in moduli ES, senza librerie né build. Due thread:



- **Game** (`main.js`) è la shell: possiede lo stato del gioco, crea un `World` per livello, lo aggiorna e lo fa disegnare.
- **World** (`world.js`) è la simulazione di un livello. Non sa nulla di schermo, suoni, punti o vite: espone il suo stato e chiama dei *hook*.
- **Renderer** (`render.js`) legge il `World` e disegna. Non lo modifica mai.
- **Audio** (`Audio` di motorino) traduce le chiamate del gioco in messaggi verso il **worklet**, che emula due chip SID e li suona.

Le parti generiche vengono da **motorino** (`lib/motorino` , submodule): loop, schermo, `Input` , font, `bake` / `drawSprite` , `parseMap` / `TileMap` , `Body` (fisica platform), `Camera` , `Particles` , `StateMachine` , `createStore` , `createI18n` , `Audio` . Vulcano tiene livelli, regole, nemici, disegno, testi, brani ed effetti; i dettagli dei moduli di motorino sono nel commento in testa a ciascuno.

Le dipendenze vanno in un solo verso (motorino escluso, che tutti usano): `main` → `world` , `difficulty` , `render` , `songs` , `sfx` ; `render` → `world` (solo lettura), `levels` , `sprites` ; `world` → `levels` , `difficulty` . Il mondo non importa niente di grafica o audio, ed è per questo che `tools/solve.mjs` può eseguirlo in Node.

2. Il ciclo principale

`startLoop` di motorino, avviato dal costruttore di `Game`, è chiamato da `requestAnimationFrame`. Usa un **accumulatore a passo fisso**:

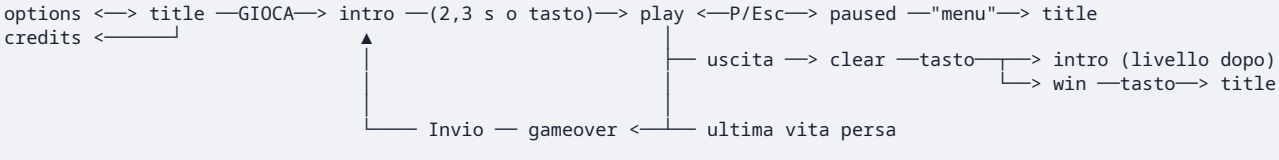
```
acc += tempo reale trascorso (max 0,25 s)
finché acc ≥ 1/60:
    input.update()     // fotografa i tasti
    game.update()      // un passo di logica
    acc -= 1/60        // (al massimo 5 passi per frame)
draw()                 // un disegno
```

Perché: tutta la fisica è tarata **per update** (es. gravità 0,4 px/update²), quindi deve girare a 60 Hz qualunque sia il refresh dello schermo. Su un monitor a 144 Hz alcuni frame non aggiornano e ridisegnano lo stesso stato; a 30 Hz si fanno due update per frame. Il limite di 0,25 s e di 5 passi evita che dopo una pausa lunga (scheda in background, breakpoint) il gioco "recuperi" correndo avanti alla cieca.

Quando la scheda si nasconde o la finestra perde il focus (`watchFocus` di motorino: `visibilitychange` e `blur`) il gioco va in pausa; l'AudioContext viene sospeso solo con la scheda nascosta.

3. La macchina a stati della shell

Una `StateMachine` di motorino (`game.states`, stati in `STATES` in fondo a `main.js`) decide cosa fa ogni update e cosa si disegna; `states.time` conta gli update dall'ingresso nello stato:



Stato	Update	Disegno
`title`	menu (↑↓ voce, ← → livello)	sfondo cielo animato, logo, menu
`options`	lingua, difficoltà, personaggio, chip, volumi; Esc torna	stesso sfondo, voci e personaggio che corre
`credits`	Invio o Esc torna	stesso sfondo, righe `creditRows` di `i18n.js`
`intro`	attende	mondo fermo + scheda del livello
`play`	`world.update(input)`; brano `golem` finché `world.bossFight`, altrimenti quello del livello	mondo + HUD
`paused`	menu pausa	mondo fermo + velo
`clear`	conta il bonus nel punteggio	tabellone
`gameover`	attende Invio / Esc	velo + GAME OVER
`win`	attende	schermata finale

Il mondo segnala gli eventi con gli hook passati al costruttore, e la shell decide cosa significano:

Hook	Chi lo chiama	Cosa fa la shell
`sfx(nome)`	ovunque nel mondo	`audio.sfx(nome)`
`onGem()`	cristallo preso	+100 punti; ogni 30 una vita (con effetto e banner)
`onStomp()`	nemico schiacciato, colpo al golem	+200 punti
`onBoss()`	golem sconfitto	+5000 punti, banner
`onDeath()`	fine dell'animazione di morte	−1 vita; respawn o game over
`onExit()`	giocatore nell'uscita	calcola bonus, sblocca il livello, salva, → `clear`

4. Livelli: dal testo alla mappa

Un livello è un array di stringhe (`levels.js`). `parseLevel(def)` lo trasforma in:

```
{ w, h,                  // dimensioni in caselle
  tiles: Uint8Array,     // w*h codici T.EMPTY, T.ROCK, T.CRUST, ...
  spawns: [{kind,x,y}],  // entità da creare (cristalli, nemici, ...)
  start: {x,y},          // casella di partenza
  gemCount }
```

Alcuni caratteri sono sia casella sia entità: `f` (bolla di fuoco) è lava *e* genera una bolla che vive in quella lava. Le bocchette dei geyser (`G`) sono caselle solide; il `World` le trova scandendo la mappa e crea un'entità geyser per ciascuna.

Il parser rifiuta caratteri sconosciuti, due partenze, nessuna partenza o nessuna uscita. Gli helper `seg` / `join` / `roof` controllano che ogni riga di un segmento sia larga uguale: l'errore tipico di un livello disegnato a mano è una riga con uno spazio in più, e così salta fuori subito con il numero di riga.

5. La simulazione (`World`)

5.1 Unità e determinismo

Pixel per le posizioni (16 per casella), pixel/update per le velocità, update per il tempo. Il mondo non legge mai un orologio: stessa sequenza di input, stessa partita. Il caso che conta per le regole (l'attesa di una bolla di fuoco fra un salto e l'altro) viene da `rand()`, un generatore del mondo (mulberry32) con il seme del livello, salvato nello stato: così una strada trovata dal solver si rigioca identica. `Math.random` resta solo nelle particelle, che non cambiano niente.

5.2 Ordine di un update

```
time++ → crolli → piattaforme → giocatore → entità → lava → particelle → camera
```

L'ordine conta: le piattaforme si muovono **prima** del giocatore, così chi ci sta sopra viene trasportato e poi collide normalmente; i pericoli si controllano **dopo** il movimento che può averli causati.

5.3 Il giocatore

Il giocatore è un `Body` di motorino con la sua taratura di default (`DEFAULT_TUNING`, che è quella di Vulcano, con il `tuning` della difficoltà sopra: vedi 5.7), sempre con la gravità in giù, e qualche campo in più (`platform`, `anim`, `dead`, `invuln`). Hitbox 10×14, più stretta dello sprite (12 px) perché un quasi-contatto sembri un mancato contatto.

Orizzontale: accelerazione verso `runSpeed` (1,9 px/update), attrito a terra, leggero smorzamento in aria.

Salto, i trucchi che lo rendono piacevole:

- **coyote time** (`coyote` = 6 update): si può saltare per un decimo di secondo dopo aver lasciato il bordo;
- **jump buffer** (`jumpBuffer` = 7): un salto premuto poco prima di atterrare parte all'atterraggio;

- **salto variabile**: rilasciando il tasto la velocità verso l'alto viene tagliata a `jumpCut`, e finché lo si tiene la gravità in salita è più bassa (`gravityUp` 0,32 contro `gravityDown` 0,4). Ne viene un salto pieno di circa 53 px (3⅓ caselle) e un saltello di circa 10 (misurati sul codice).

Collisioni (`Body.step` contro la `TileMap` del mondo): un asse per volta, x poi y. Si sposta il box e lo si ferma contro la prima casella solida che incontra. Le velocità restano sotto i 16 px/update, quindi il box non può attraversare un muro in un passo.

Crosta (`=`): casella *one-way* della `TileMap`, solida solo se il fondo del box era sopra la superficie *prima* del movimento. ↓ + salto sopra una crosta imposta `dropThrough` per qualche update e si cade attraverso.

Piattaforme mobili: quando il giocatore atterra su una piattaforma, `player.platform` punta a lei. In `updatePlatforms` la piattaforma sposta il giocatore dello stesso delta. Il `Body` non vede le piattaforme (non sono caselle): dopo il suo passo, `ridePlatforms` tiene il giocatore agganciato finché è dentro i bordi e non salta. La regola "era sopra" è allargata dello spostamento della piattaforma, così un ascensore che sale "raccoglie" i piedi in cui entra.

Pericoli (`checkHazards`): lava e spuntoni contano solo quando i piedi ci sono dentro di qualche pixel, per perdonare chi sfiora la superficie. Nemici, bolle e geyser si controllano nei rispettivi update.

Morte e respawn: `kill` avvia un'animazione di 80 update (salto all'indietro, esplosione di scintille, scossa della camera). Alla fine il mondo chiama `onDeath`; se la shell sceglie il respawn, `respawn()` rimette il giocatore all'ultimo checkpoint con un secondo di invulnerabilità.

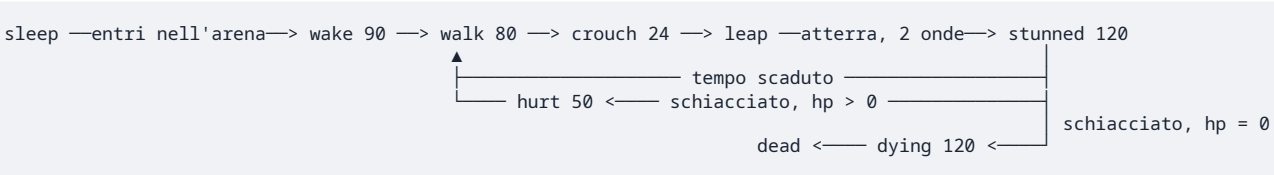
5.4 Le entità

Oggetti semplici con un campo `kind`; ognuno ha la sua funzione di update e il suo pezzo di disegno.

Kind	Comportamento
`gem`	fermo; preso al contatto
`checkpoint`	al contatto diventa attivo e sposta `respawnPoint`
`exit`	al contatto chiama `onExit`; `locked` (con un golem nel livello) la tiene chiusa
`boss`	il golem: vedi sotto
`wave`	onda d'urto del golem: corre sul pavimento fino al bordo dell'arena
`slug`	cammina, gira contro i muri e **sull'orlo** (non cade mai in lava)
`bat`	ignora la mappa: un otto attorno al punto di nascita
`bubble`	aspetta nella lava, viene lanciata a ~72 px (`v = √(2gh)`), ricade
`geyser`	ciclo di 200 update: 100 quiete, 40 preavviso (fumo, bocchetta che lampeggia), 60 getto alto 48 px
`platform`	`M` orizzontale / `N` verticale, rimbalza su roccia o fermi `|`

Schiacciare: un contatto con un nemico è uno schiacciamento se il giocatore scende e il suo fondo, nell'update prima, era sopra la testa del nemico (+4 px di tolleranza). Il rimbalzo è pieno se si tiene il salto.

Il golem (`updateBoss`) è una macchina a stati (durate in update):



Se il giocatore muore il golem torna in `sleep` , a casa, con gli hp che aveva; da `dead` apre le uscite e chiama `onBoss` .

Nel salto mira al punto dove sta il giocatore alla fine dell'accucciata: il tempo di volo è fisso (salita a 5,5 px/update, gravità 0,22, 50 update), quindi la velocità orizzontale è distanza / tempo. Da caldo (`walk` , `crouch` , `leap` , `wake`) ogni contatto brucia; da freddo (`stunned` , `hurt`) è roccia: il contatto laterale non fa niente e lo schiacciamento, solo in `stunned` , toglie un hp. `rage = maxHp − hp` accelera camminata (0,5 + 0,3·rage), onde (1,8 + 0,4·rage) e accorcia lo stordimento (120 − 20·rage); tutti e tre vanno poi moltiplicati o divisi per il `boss` della difficoltà (5.7).

Dal risveglio alla caduta (`bossFight`) l'arena è sigillata: il giocatore non può tornare a sinistra del suo bordo (la cortina di lava) e la camera resta dentro l'arena. Senza, si vince stando fuori portata: il solver l'ha trovato subito. Il golem si cerca ogni volta in `entities` (`get boss()`) invece di tenerne un riferimento, perché il solver ripristina `entities` per copia.

Rocce che crollano: al primo contatto dall'alto parte un conto alla rovescia di 28 update (la casella trema sempre di più), poi la casella diventa vuota per 240 update e ricresce — a meno che il giocatore non sia lì, nel qual caso aspetta ancora.

5.5 La lava che sale

Nei livelli con `rise` la lava è una linea `lavaY` sotto il livello. Dopo `rise.delay` update (con rombo e scossa) sale a `rise.speed` px/update **più** `0,003 × (distanza − 150)` se il giocatore è più di 150 px sopra. È un elastico: chi indugia viene punito, chi sale veloce si trova comunque la lava alle calcagna. Al respawn la lava viene riportata 9 caselle sotto il checkpoint e fermata per 4 s, altrimenti una morte in alto farebbe rinascere dentro la lava.

Nel condotto: 300 update (5 s) di attesa, poi 0,13 px/update, circa mezza casella al secondo. Era 200 e 0,28 (una casella al secondo): il solver passava, ma per una persona che deve ancora imparare il livello era troppo, e anche 0,18 lo era ancora.

5.5 bis La lava che insegue

Nei livelli con `chase` la lava è una linea verticale `lavaX` a sinistra della mappa. Dopo `chase.delay` update (rombo e scossa) avanza a `chase.speed` px/update, più `0,003 × (distanza − 240)` se il giocatore è più di 240 px avanti. Uccide quando il bordo sinistro del giocatore la supera di 3 px. Al respawn torna 8 caselle a sinistra del checkpoint e si ferma per 4 s.

Nella colata: 300 update di attesa, poi 0,5 px/update (meno di due caselle al secondo, contro le sette della corsa). Il solver la finisce anche a 1,6 perché non si ferma mai: il margine serve a chi si ferma per i cristalli e sbaglia qualche salto. Il primo valore, 0,9, era troppo per una persona.

5.6 Camera e particelle

La camera (`Camera` di motorino) insegue un bersaglio (centro del giocatore + 24 px nella direzione in cui guarda, 10 px sopra il centro) con un'interpolazione esponenziale (10% per update in x, 12% in y), ed è limitata ai bordi della mappa (durante lo scontro col golem, a quelli dell'arena). Durante la morte si ferma.

Le particelle (`Particles` di motorino) sono puramente estetiche: posizione, velocità, gravità, vita, colore, e un flag `glow` per il disegno additivo. Massimo 400. Il solver sostituisce `particle()` con una funzione vuota.

5.7 La difficoltà

`game/difficulty.js` tiene una tabella con tre voci (`easy` , `normal` , `hard`); la shell passa quella scelta al `World` come `diff` , accanto a `god` . `normal` è il gioco come tarato nelle prove; le altre lo scalano:

Campo	Chi lo usa	Facile	Normale	Difficile
`lives`	shell: vite iniziali e al riprova	9	5	3
`gemsPerLife`	shell: cristalli per una vita	20	30	50
`lava`	`rise.speed`, `chase.speed` ×	0,6	1	1,3
`lavaDelay`	`rise.delay`, `chase.delay` e attesa al respawn ×	2	1	0,6
`enemies`	passo delle lumache e dei pipistrelli ×	0,6	1	1,3
`bossHp`	colpi del golem	2	3	4
`boss`	camminata e onde ×, stordimento ÷	0,7	1	1,2
`tuning`	sopra la taratura del `Body`	`coyote` 10, `jumpBuffer` 12		

Il salto (altezza, lunghezza, velocità di corsa) non cambia mai: i livelli sono costruiti attorno a lui. Per i bambini il facile allarga invece le due tolleranze (salto dal bordo per 10 update invece di 6, salto in anticipo 12 invece di 7). La lava facile nel condotto sale a 0,078 px/update dopo 10 s, nella colata insegue a 0,3; la difficile a 0,169 e 0,65 dopo 3 s. Il solver prova ogni livello a ogni difficoltà: anche il facile, perché nemici più lenti cambiano i tempi su cui i livelli erano stati provati.

6. Il disegno (`Renderer`)

Tutto si disegna su un canvas **320×180** (16:9, 20×11 caselle). Il CSS lo scala di un fattore intero con `image-rendering: pixelated`, così i pixel restano quadrati e netti; sotto lo zoom 2× usa la scala esatta.

Ordine dei livelli per frame:

1. sfondo in parallasse (`sky`: cielo a gradiente, vulcano lontano con bagliore pulsante e pennacchio di fumo, colline, cenere che cade; `cave`: texture di roccia, colate di lava lontane che tremolano);
2. **strato statico** del livello, già pre-renderizzato (vedi sotto), copiato solo per la finestra visibile;
3. caselle animate: lava (corpo con macchie che scorrono, superficie a onda) e rocce che crollano (con tremolio);
4. bagliore additivo sulle superfici di lava;
5. entità; 6. giocatore; 7. lava che sale; 8. particelle;
6. (dalla shell) HUD e schermate sovrapposte.

Pre-rendering: `bakeLevel` disegna una volta, all'inizio del livello, roccia, crosta, spuntoni e bocchette in un canvas grande quanto tutto il livello. Le rocce cambiano aspetto in base ai vicini (bordo superiore illuminato, lato inferiore con stalattiti, ombre laterali, vene di lava rare sulle superfici esposte). Farlo a ogni frame costerebbe decine di migliaia di `fillRect`. La texture usa un hash deterministico delle coordinate, così il livello ha sempre lo stesso aspetto.

Luce: non c'è un motore di luci. Il bagliore è un gradiente radiale pre-calcolato disegnato con `globalCompositeOperation = 'lighter'` (somma additiva): sopra la lava, attorno a cristalli, bolle, geyser, checkpoint attivi e uscita.

Sprite (`game/sprites.js`, cotti da `bake` di motorino): disegnati come testo, un carattere per pixel con una palette, cotti all'avvio in piccoli canvas. Il giocatore ha un busto comune e cinque varianti di gambe (fermo, tre passi di corsa, salto). I personaggi (`CHARACTERS`: esploratore, robottino, ragno, riccio) sono solo pelli: stesso riquadro 12×16 e stessa hitbox, la fisica non li distingue. Il robottino e il riccio riusano le gambe dell'esploratore ricolorate; il ragno, visto di fronte, ha gambe sue. `Renderer.character` sceglie quale disegnare. Tutti gli sprite guardano a destra; `drawSprite` li specchia e li ancora per il centro dei piedi.

Testo (`font.js` di motorino): font bitmap 5×7 disegnato con `fillRect`, così resta nitido a ogni scala. Ogni glifo è precompilato in "strisce" orizzontali (pochi `fillRect` invece di 35). Le lettere accentate si riducono alla lettera base, tranne quelle italiane, che hanno un glifo proprio (À È É Ì Ò Ù: in italiano "è" ed "e" sono parole diverse), con l'accento disegnato nelle due righe sopra la cella. Il colore può essere un gradiente (titoli "di fuoco").

Lingue (`i18n.js`): tutti i testi a schermo, in italiano e inglese, per chiave (`this.t.play`); quelli con un numero sono funzioni (`t.lives(3)`). I nomi dei livelli stanno in `levels.js` come `{ it, en }`. Una nuova lingua è un blocco in più in `STRINGS`: il menu la offre da sé.

7. L'input (`Input` di motorino)

Tastiera, gamepad e touch diventano un insieme di **azioni**: `left`, `right`, `up`, `down`, `jump`, `start`, `pause`, `mute`, `fullscreen`. Una volta per update `input.update()` costruisce l'insieme `now` e conserva il precedente in `prev`:

- `down(a)` — l'azione è tenuta;
- `pressed(a)` — è stata premuta in questo update (`now` e non `prev`).

Un tocco rapidissimo (giù e su tra due update) andrebbe perso: per questo i `keydown` finiscono anche in un insieme `hits` che viene svuotato a ogni update, e conta come pressione anche se il tasto è già stato rilasciato.

Lo schermo intero il browser lo concede solo durante un gesto: per questo non si chiede nell'update ma in `input.onPress`, chiamato dentro l'evento del tasto o del tocco (F, oppure conferma sulla riga SCHERMO INTERO delle opzioni).

La shell avvia l'audio appena caricata, così la musica del titolo suona subito dove il browser lo consente. Di solito non lo consente: senza un gesto l'AudioContext nasce sospeso. Chrome fa eccezione se si ricarica un tab già toccato (prova fatta: `running` ricaricando, `suspended` in un tab nuovo). Il primo gesto dell'utente (tasto, clic o tocco) chiama `onFirstGesture`, che richiama `start()` e lo riprende.

8. L'audio

8.1 Perché un AudioWorklet

L'audio va generato campione per campione (44 100 o 48 000 al secondo). L'AudioWorklet gira in un thread audio separato con buffer di 128 campioni, e il disegno o il garbage collector del thread principale non lo interrompono. Il main thread non genera mai un campione: manda messaggi.

```
main thread (Audio)                              audio thread (sidicio/src/worklet.js)
  start() — createSidicioNode(ctx) ————————————> SidicioProcessor
  post {songs}, {sfxDefs}, {gain}, {model}  ——>  configura
  play('magma') —— {type:'play'} ————————————>   MusicPlayer.play(song)
  sfx('jump')   —— {type:'sfx'}  ————————————>   SfxPlayer.play(def)
                                  <————————      {type:'songEnd'}
```

8.2 Il chip (`SidChip`)

Sta in sidicio (`lib/sidicio/src/chip.js`), libreria separata inclusa come submodule; i player sono in `music.js` e `sfx.js` dello stesso pacchetto, il compilatore dei brani in `song.js`. Port JavaScript del motore C++ di mcma-chip (`native/sid_chip.cpp`), scritto dal datasheet e dalle note di reverse engineering pubbliche, non da reSID. Interfaccia "a registri": frequenza (16 bit), larghezza d'impulso (12 bit), forma d'onda, ring, sync, ADSR, gate per voce; filtro e volume globali.

Oscillatori: un accumulatore di fase a 24 bit per voce, avanzato di `freq` per ciclo di clock del C64 PAL (985 248 Hz). Le onde si ricavano dai bit dell'accumulatore come nel chip vero:

- **dente di sega** = i 12 bit alti;
- **triangolo** = l'accumulatore "ripiegato" sul bit più alto (con ring mod, quel bit è messo in XOR con quello della voce sorgente);
- **impulso** = confronto dei 12 bit alti con la larghezza d'impulso;
- **rumore** = un LFSR a 23 bit cadenzato dal bit 19 dell'accumulatore; 8 suoi bit formano l'uscita.

Più onde insieme si combinano in AND, come approssimazione delle onde combinate del chip. **Sync**: quando il bit alto della voce sorgente sale, l'accumulatore della voce sincronizzata si azzera. Sorgenti: voce 1 ← 3, 2 ← 1, 3 ← 2.

Inviluppo: attacco lineare a 256 passi; decadimento e rilascio passano per un secondo divisore il cui periodo cresce al calare del livello (1, 2, 4, 8, 16, 30): è la coda quasi-esponenziale che fa "suonare" un rilascio SID. I periodi in cicli per i 16 valori di velocità sono quelli del datasheet.

Filtro: state-variable a trasformata TPT (stabile fino all'alto), con uscite passa-basso, passa-banda e passa-alto sommabili. Le curve cutoff→Hz sono due: **8580** quasi lineare 30 Hz–12 kHz con risonanza forte; **6581** esponenziale 200 Hz–18 kHz, risonanza più dolce e saturazione morbida all'ingresso. Ogni voce può essere instradata o no nel filtro.

Oversampling 4×: gli oscillatori SID sono rampe e impulsi "ingenui"; generati direttamente a 48 kHz, le loro armoniche si ripiegano in fischi inarmonici. Il chip calcola quattro campioni interni per ogni campione d'uscita e li filtra con un Butterworth del 4° ordine (due biquad) prima di decimare. Infine un DC blocker (un impulso stretto o un sustain fermo sposterebbero la membrana dell'altoparlante) e un soft clip.

Il codice JS segue il C++ riga per riga, con due adattamenti: la moltiplicazione a 64 bit della fase usa i `double` (esatti fino a 2^{53}), e i flag di sync si leggono tutti prima di azzerare, così un azzeramento non nasconde il fronte di un altro.

8.3 Due chip

Un C64 ha un solo SID, e nei giochi veri un salto "ruba" per un attimo una voce alla musica. Qui ci sono due chip: `musicChip` per la musica e `sfxChip` per gli effetti, sommati con guadagni separati. La musica non perde mai una nota, e i volumi di musica ed effetti si regolano indipendentemente.

8.4 Il tick a 50 Hz

Nel `process` del worklet un contatore scende di uno a ogni campione; ogni `sampleRate / 50` campioni chiama `music.tick()` e `sfx.tick()`. È l'equivalente del player che un musicista C64 agganciava all'interrupt del raster PAL: una volta per frame si scrivono i registri, poi il chip suona da solo fino al frame dopo. Arpeggi, vibrato, sweep e batteria nascono tutti da cosa si scrive a ogni tick.

8.5 Il player musicale (`MusicPlayer`)

Un brano compilato è:

```
{ speed,                    // tick per riga
  rows: [Int32Array × 3],   // una cella per riga per voce
  loopRow,                  // riga a cui tornare, -1 = fine
  instruments: [...],       // gli strumenti usati
  filter }                  // cutoff, risonanza, modo, LFO
```

Una cella è `HOLD` (−1), `OFF` (−2) o `nota | strumento << 7 | glide << 14`.

Ogni `speed` tick si legge una riga: per ogni voce una nuova nota imposta lo strumento, l'ADSR e **riattacca** l'inviluppo (gate giù e su nello stesso frame, come un player C64); `OFF` chiude il gate; una nota con glide cambia solo l'altezza di arrivo. A ogni tick, per ogni voce attiva si calcolano altezza, onda e larghezza d'impulso dallo strumento e si scrivono nel chip. Il cutoff del filtro può essere modulato da un LFO a coseno su tutto il brano (il "respiro" dei pad e dei bassi).

Strumenti (`INSTRUMENTS` in `songs.js`):

Campo	Significato
`wave`	forma d'onda (`TRI` , `SAW` , `PULSE` , `NOISE` , combinabili con `|`)
`adsr`	`[attacco, decadimento, sustain, rilascio]` , 0–15 come nel chip
`pw` , `pwSpeed` , `pwMin` , `pwMax`	larghezza d'impulso iniziale e sweep avanti-indietro (il classico suono "che si muove" del SID)
`arp` , `arpSpeed`	arpeggio: semitoni sommati a turno, un passo ogni `arpSpeed` tick — gli accordi suonati da una voce sola
`vib`	`[ritardo, profondità in semitoni, velocità]`
`glide`	tick per scivolare verso una nota con `~`
`table`	wave table: per ogni tick `[onda, nota, assoluta?]` ; serve alla batteria (un frame di rumore, poi un impulso che scende di altezza = cassa)
`filter`	la voce passa nel filtro
`ring` , `sync`	modulazione ad anello / sync con la voce precedente

Compilatore (`compileSong`): unisce le sezioni, converte i token in celle, numera gli strumenti usati, segna la riga di loop. Ogni voce parte con uno strumento predefinito (basso, solista, accordi) se la stringa non ne nomina uno. La batteria condivide la terza voce con gli accordi arpeggiati: un colpo di cassa interrompe l'accordo, che riparte dopo — come in molti brani C64.

I brani:

Nome	Dove	Carattere
`title`	titolo	Re minore, lento: campana su pad filtrato, basso lungo
`magma`	livelli 1–2	La minore con Si♭ frigio, 125 BPM, basso a ottave, solista impulso/sega
`fuga`	livelli 3–5	Mi minore, 150 BPM, basso al galoppo, filtro che "respira" veloce
`golem`	scontro col golem	Do minore, 150 BPM, i–VI–VII–V con Re♭ frigio, basso martellante, cassa in levare, filtro molto risonante
`clear`	fine livello	fanfara breve, non ripete
`gameOver`	game over	discesa cromatica, non ripete
`victory`	vittoria	fanfara, poi la shell rilancia il titolo

8.6 Il player degli effetti (`SfxPlayer`)

Un effetto è un **programma a frame**: una voce `[onda, hz, larghezzaImpulso]` per tick. `null` chiude il gate per un tick; un'onda con il bit `RETRIGGER` riattacca l'inviluppo (note staccate dentro un effetto). Un `cutoff` opzionale manda la voce nel filtro con un cutoff che cambia nel tempo (lo sfrigolio della lava, il soffio del geyser).

Tre voci, quindi tre effetti insieme. Un quarto prende la voce libera o, se non ce n'è, la più vecchia con priorità uguale o minore; se tutte sono più importanti l'effetto non suona. Così il suono della morte (priorità 4) non viene mai coperto da un cristallo (1).

8.7 Il lato main thread (`Audio`)

- `start()` — al caricamento: crea l'AudioContext, carica il modulo, crea il nodo, manda brani compilati ed effetti, applica le impostazioni e fa partire il brano richiesto nel frattempo. Richiamato al primo gesto, riprende il contesto se il browser l'ha lasciato sospeso.
- `started` — vero solo se il contesto suona davvero (`running`): finché è falso il titolo mostra "PREMI UN TASTO".
- `play(nome)` non fa nulla se quel brano sta già suonando (passare dal livello 1 al 2 non riavvia "magma"); `replay(nome)` riparte sempre.
- `applySettings()` — volumi al quadrato (lo slider a 3 è davvero basso), muto, modello di chip.
- Pausa: il player musicale chiude i gate e smette di avanzare; gli effetti continuano (il suono della pausa stessa).

9. Gli strumenti di verifica

`tools/check.mjs` importa i moduli audio (quelli di sidicio e i brani del gioco) e i livelli in Node (il worklet di sidicio salta `registerProcessor` se non esiste), rende ogni brano e ogni effetto con lo stesso codice del gioco e controlla `NaN`, silenzio, lunghezze delle sezioni e fine dei jingle.

`tools/solve.mjs` è la prova che i livelli si possono finire:

- crea un `World` vero con hook finti, una volta per difficoltà;
- lo stato mutabile (giocatore, entità, caselle, crolli, le due lave, tempo, seme del generatore) si salva e ripristina con `structuredClone`, che preserva anche il riferimento giocatore → piattaforma; il clone perde le classi, quindi giocatore (`Body`) e camera si ricopiano nelle istanze vive invece di sostituirle;
- l'input è un `FakeInput` di motorino; mondo, azioni e salvataggio stanno in `tools/sim.mjs`, condiviso con i test;
- ogni nodo della ricerca prova 6 azioni (sinistra/ferma/destra × salto sì/no) tenute per 6 update; le morti si scartano;
- gli stati si deduplicano su posizione (3 px), velocità, a terra, e tempo a blocchi;
- si espande sempre il nodo più vicino all'uscita (best-first), fino a 250 000 nodi;

- la strada trovata si scrive in `test/replays/level<N>.<difficoltà>.json` (azioni per nome: `L`, `-`, `R`, con `+J` per il salto) e `npm test` la rigioca dall'inizio del livello. I test durano un attimo; la ricerca, circa due minuti in tutto, gira solo quando serve.

Con un golem l'obiettivo cambia per fasi: prima raggiungere l'arena; dentro, ogni hp rimasto pesa più di qualunque distanza, tra un colpo e l'altro conta **sopravvivere più a lungo** (le finestre di stordimento arrivano a chi aspetta) e, mentre il golem è stordito, stargli vicino alla testa; caduto il golem, l'uscita. In più un "cricchetto": appena un ramo mette a segno un colpo, i rami in cui il golem ha più hp vengono scartati. Con la sola distanza dal golem la ricerca restava incollata a lui senza trovare il secondo colpo.

Il blocco di tempo è il compromesso: fine (1 s) segue la fase delle piattaforme mobili; grossolano (3 s) tiene la ricerca piccola nei livelli lunghi. Il solver prova il primo e, se fallisce, il secondo.

Il solver ha già trovato tre errori veri durante lo sviluppo: un ascensore che ignorava il suo fermo perché lo cercava solo sotto il centro, un pipistrello che chiudeva l'unico salto possibile nel condotto, e un golem che si batteva aspettando fuori dall'arena.

10. Scelte e limiti

- **Niente motore, niente librerie.** Il gioco è piccolo; un motore avrebbe portato più codice di quanto ne toglie, e il SID va scritto comunque a mano.
- **Livelli come testo.** Si leggono e si modificano in un diff, e il solver li verifica. Il prezzo: niente editor visuale.
- **Onde combinate approssimate** con l'AND: sul chip vero sono più deboli e sporche (6581 soprattutto). Lo stesso compromesso di mcma-chip.
- **Audio ~12% di un core.** Vedi TECNICA.md.
- **Nessun salvataggio a metà livello**: si salvano solo livello sbloccato, record e impostazioni.